

1. Práctica 11: Comunicación remota con *ThingsBoard*

1.1. Objetivos

- Entender y configurar la plataforma de *ThingsBoard*.
- Programar el STM32F723 para comunicarse con *ThingsBoard*, enviando y recibiendo datos.
- Entender los protocolos de comunicación como MQTT.

1.2. Fundamentos teóricos

En general, cualquier plataforma empotrada hoy en día tiene cierta conectividad con la red. Desde electrodomésticos inteligentes hasta estaciones de monitorización atmosférica, pasando por bombillas que pueden cambiarse de color desde el móvil, todos estos dispositivos pueden enviar y recibir información desde internet.

Para ello existen numerosos protocolos. Uno de los más extendidos, por su ligereza y versatilidad, es **MQTT**. **MQTT** es un protocolo ligero para el intercambio de mensajes, diseñado para la comunicación en redes con ancho de banda limitado, y por dispositivos con capacidad de computación reducida. Debido a su simplicidad y escalabilidad, MQTT es de particular interés para aplicaciones de Internet de las cosas (IoT).

MQTT utiliza un formato de mensaje binario (Figura 1), que reduce enormemente la cantidad de información transmitida frente a otros protocolos basados en texto como **HTTP**.

MQTT Control Packet Hierarchy

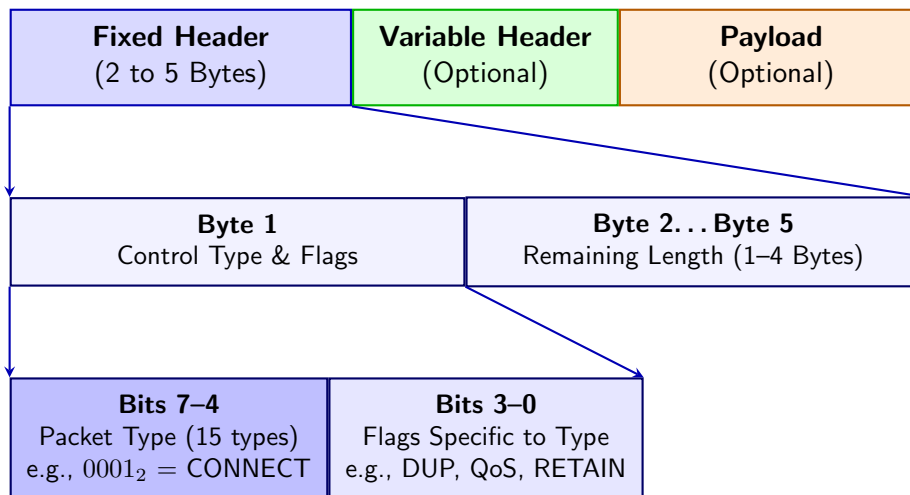


Figura 1: Formato de los paquetes **MQTT**

En **MQTT** se distinguen los *clientes* y los *brokers*. Un cliente puede enviar mensajes al broker (conteniendo generalmente actualizaciones de valores etiquetados), y el broker reenvía estas actualizaciones a otros clientes que estén suscritos a dichos valores etiquetados. Las etiquetas se indican categorizadas, con cada categoría separada por “/”. Así, se permiten suscripciones a categorías completas, por ejemplo (luego lo veremos en más detalle al hacer la práctica). Un ejemplo sencillo de esta estructura puede verse en Figura 2

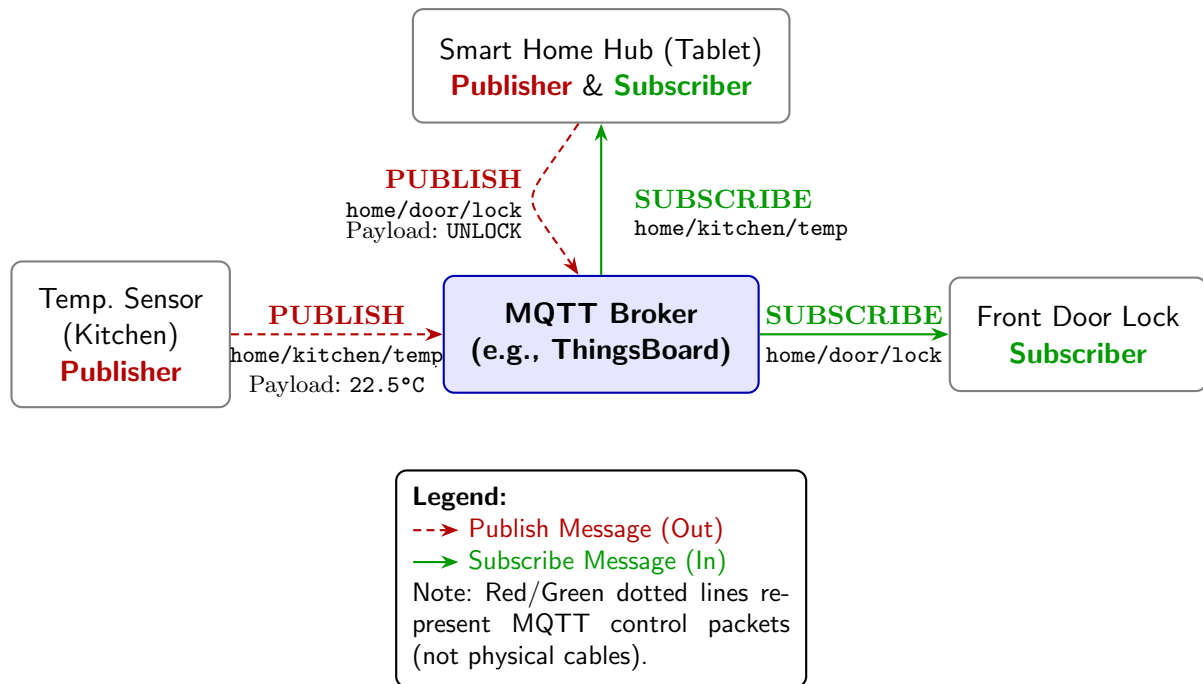


Figura 2: Generic MQTT diagram

1.2.1. El módulo Wifi ESP-01S

Nuestro procesador no tiene manera de realizar comunicación inalámbrica, por tanto se utiliza un módulo externo ESP-01S conectado al conector CN14. Este módulo permite, a través de una comunicación **UART** desde el procesador **STM32F723IE**, establecer una conexión WiFi y enviar o recibir datos a través de protocolos como HTTP y TCP, que serán vistos desde nuestra aplicación a través de la **UART**.

Nosotros lo veremos solo como usuarios y con un conjunto reducido de comandos. Puedes consultar la documentación completa en https://espressif-docs.readthedocs-hosted.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Set/index.html.

1.3. Desarrollo de la práctica

Para realizar la práctica, vamos a hacer uso de un *broker* llamado **ThingsBoard** (<https://thingsboard.io/>). Es una plataforma online que ofrece, de manera gratuita, un *broker* para hasta 5 dispositivos. Como solo tenemos uno, será suficiente.

Para nuestro cliente, utilizaremos las librerías de Eclipse Paho MQTT (<https://github.com/eclipse-paho/paho.mqtt.embedded-c>), una implementación ligera de MQTT para dispositivos empujados.

1.3.1. Creando cuenta y dispositivo en *ThingsBoard*

Lo primero de todo es crear una cuenta gratuita en *Thingsboard Cloud Europe* (<https://eu.thingsboard.cloud/signup>). Veremos algo similar a la Figura 3 al iniciar.

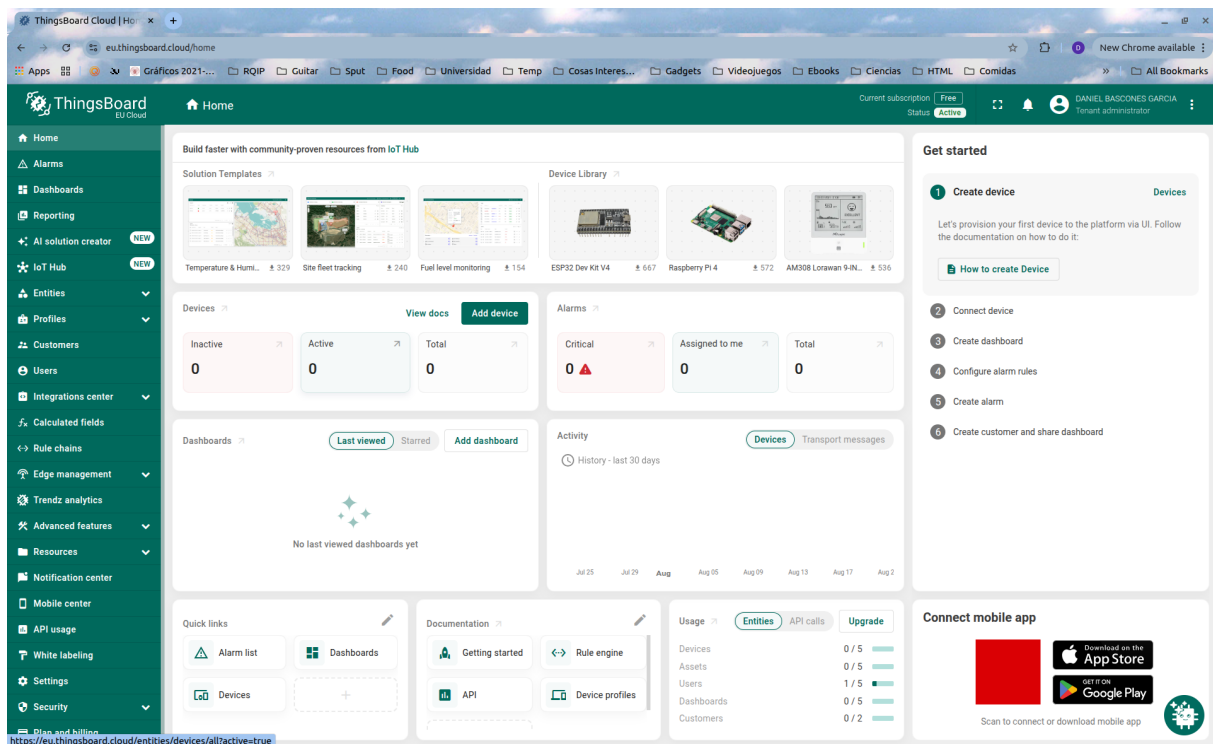


Figura 3: Pantalla inicial de *ThingsBoard* tras iniciar sesión

A continuación, tendremos que crear un dispositivo (desde el panel “Devices”) (que representará nuestra placa **32F723EDISCOVERY**) como en la Figura 4.

Para conectarnos posteriormente, necesitaremos identificarnos (y demostrar, desde el dispositivo, que tenemos derecho a enviar/recibir información). Aunque hay varias maneras de hacerlo, nosotros lo haremos con un *token* de acceso. Para ello, entra en la configuración del dispositivo y busca el *token* dentro de *credentials* (Figura 5). Anótalo bien, lo necesitarás.

1.3.2. Programando nuestro dispositivo para conectar

En esta práctica no vas a tener que programar demasiadas cosas, ya que la complejidad de las librerías es elevada. La base del código, ya funcional, está hecha, y tiene esta pinta:

```
thingsboard_data_control/
├── Src/..... – Source code directory
│   └── main.c..... – Core application entry point
├── Inc/..... – Include files
│   ├── bsp.c..... – Board Support Package Code
│   ├── bsp.h..... – Board Support Package Header
│   ├── ESP01S_config_template.h..... – Template to fill with config constants
│   ├── ESP01S.c..... – ESP01S Controller Code
│   ├── ESP01S.h..... – ESP01S Controller Header
│   ├── MQTT*..... – Eclipse Paho Embedded MQTT Library
│   ├── RingBuffer.c..... – Helper Receiver Buffer Code
│   ├── RingBuffer.h..... – Helper Receiver Buffer Header
│   └── StackTrace.h..... – Eclipse Paho Embedded MQTT Library
```

Lo único que falta es renombrar el archivo `ESP01S_config_template.h` por `ESP01S_config.h`,

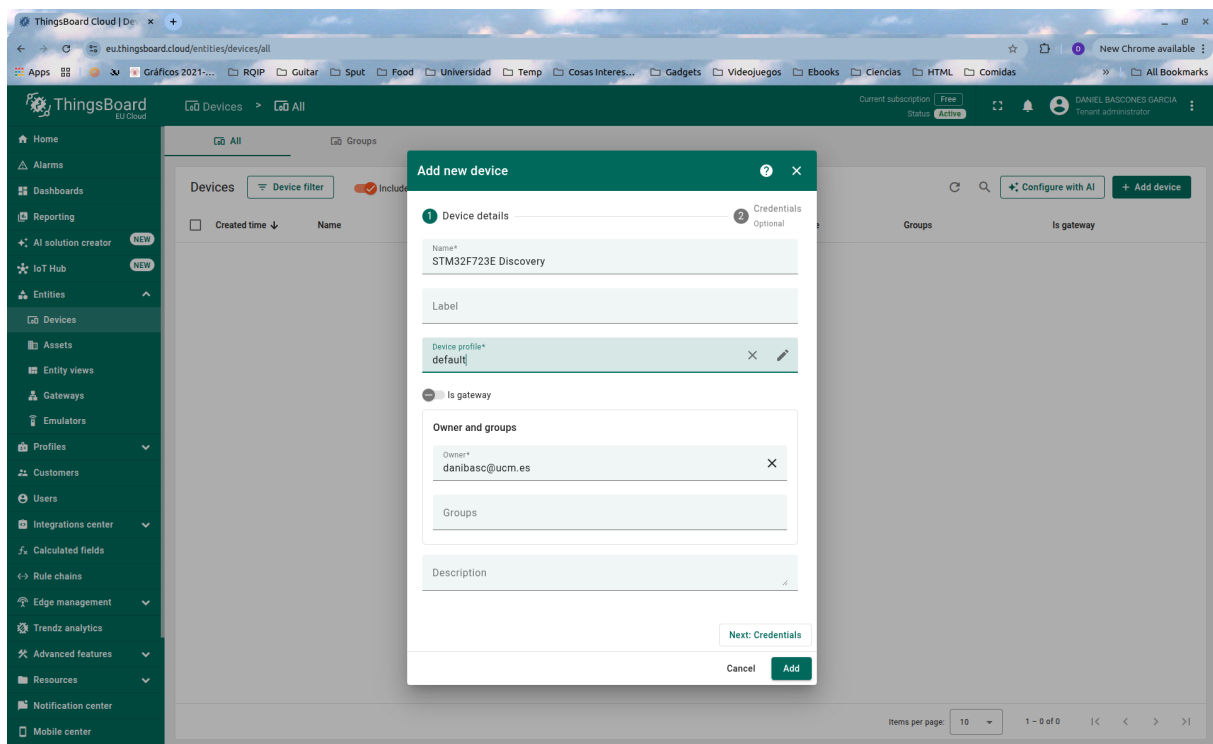


Figura 4: Pantalla de añadir un nuevo dispositivo

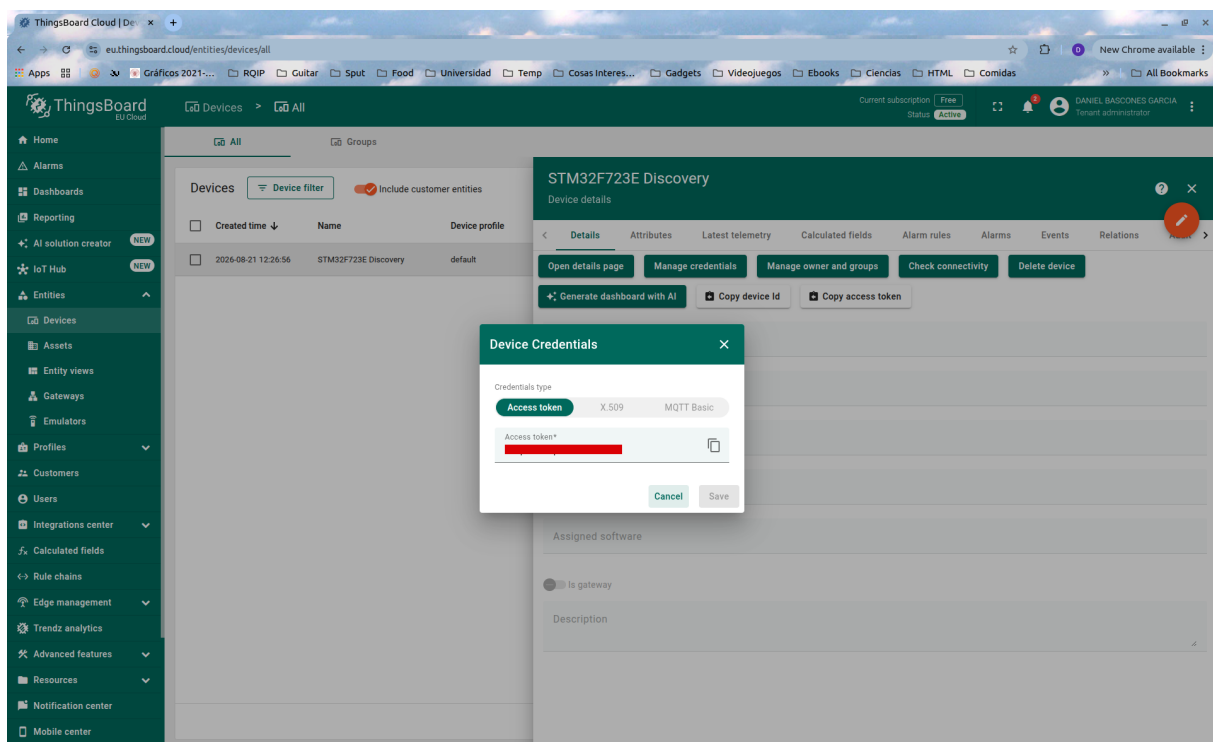


Figura 5: Credenciales de nuestro dispositivo

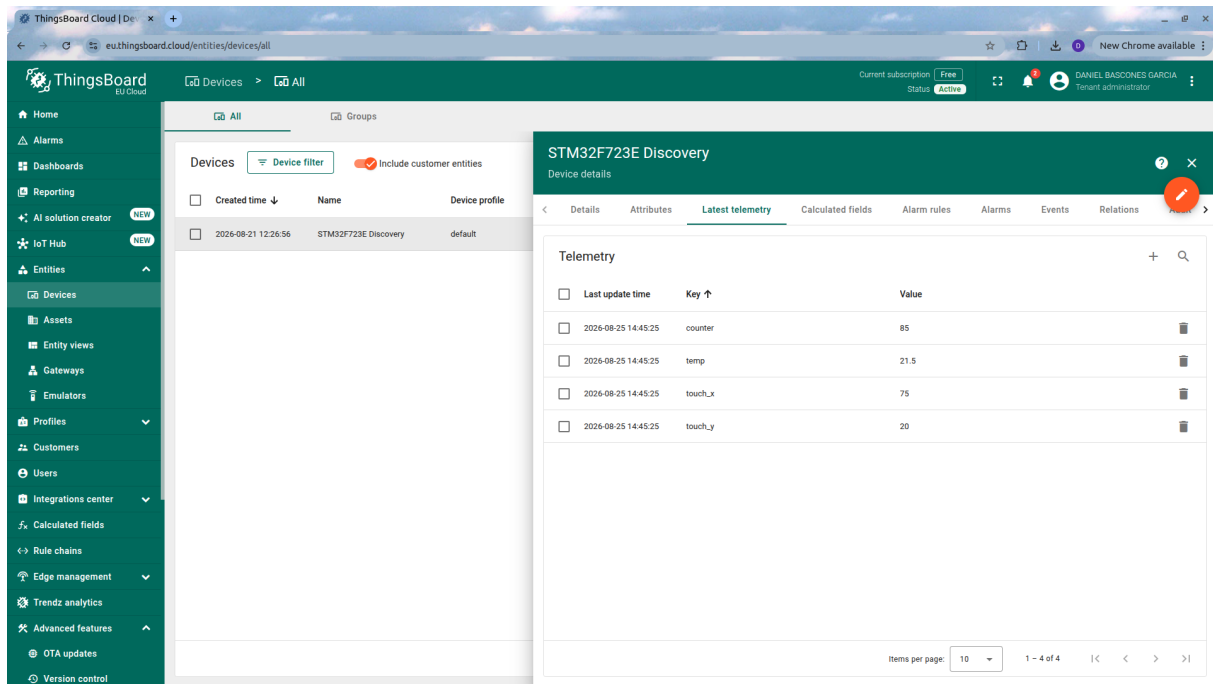


Figura 6: Telemetría vista desde thingsboard

y configurar las credenciales del punto WiFi a utilizar (puede ser vuestro teléfono en modo AP) y el *token* que hemos obtenido anteriormente.

Al lanzar ahora la aplicación, deberíamos ver por el terminal (recuerda abrirlo con *picocom*) los siguientes mensajes de depuración):

```

1 [ESP] Testing ESP01S AT Communication...
2 [ESP] Disabling Echo...
3 [ESP] Setting Station Mode...
4 [ESP] Disabling MUX (Single Conn Mode)...
5 [ESP] Enabling Transparent Passthrough Mode...
6 [ESP] Connecting to SSID: WIFI_SSID ..
7 [MQTT] Connecting to eu.thingsboard.cloud:1883...
8 [MQTT] Entering passthrough streaming...
9 [MQTT RX] CONNACK 0x00: Connection Accepted!
10 [MQTT RX] SUBACK Received! Granted QoS: 0
11 [MQTT] Connected & Subscribed successfully!
12 [SYSTEM] Ready! Streaming over MQTT...
13 [MQTT TX] {"temp":20.50,"counter":1,"touch_x":15,"touch_y":20}
14 [MQTT TX] {"temp":21.00,"counter":2,"touch_x":30,"touch_y":40}
15 [MQTT TX] {"temp":21.50,"counter":3,"touch_x":45,"touch_y":60}
16 [MQTT TX] {"temp":22.00,"counter":4,"touch_x":60,"touch_y":80}
17 [MQTT TX] {"temp":22.50,"counter":5,"touch_x":75,"touch_y":100}

```

Con esto, hemos establecido la conexión al *broker* de **MQTT** de *ThingsBoard*. Algunas cosas a notar es que, debido a la simplicidad de esta implementación (si echas un vistazo al código, verás que se implementa un cliente “a mano”), no tenemos control de errores ni QoS superior a 0. Es decir, que si algún mensaje se pierde, se ha perdido y no se recupera.

De momento, en cualquier caso, estamos enviando mensajes al servidor con varias variables (en este caso, creadas de manera sintética). Si entramos en *ThingsBoard*, dentro de la gestión del dispositivo, deberíamos ver ya las variables de telemetría como en la ??.

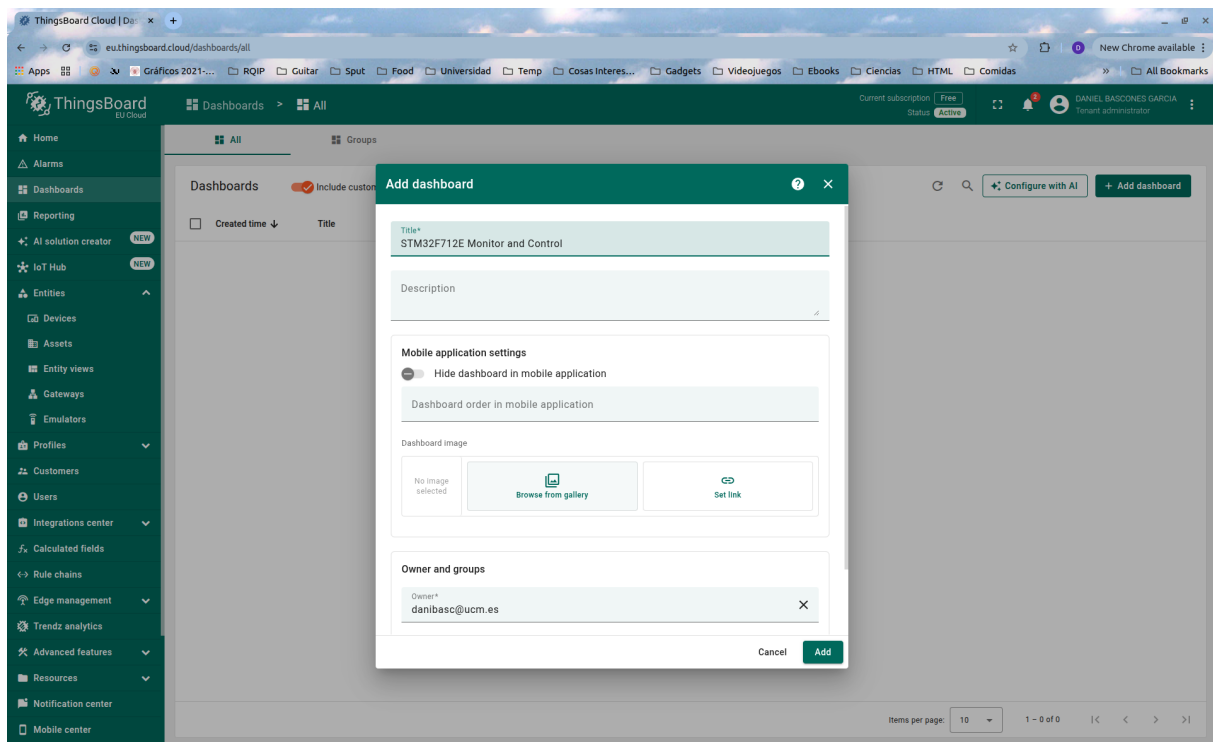


Figura 7: Creación de un panel de control

1.3.3. Visualización en *ThingsBoard*

ThingsBoard tiene un sistema de visualización muy potente para crear gráficas, así que vamos a aprovecharlo, para ver bien estos datos, en lugar de en una lista numérica. Lo primero para poder utilizarlo es crear un *Dashboard* como en la Figura 7.

Ahora podemos añadir fácilmente elementos a nuestro panel desde el modo edición ¿añadir widget. Por ejemplo una gráfica que muestre la temperatura como en la Figura 8. Asegúrate muy bien de que el nombre de la variable visualizada es el mismo que el que da nuestro dispositivo. A veces por defecto sale solo “temp” y no funcionará si es así.

También puedes añadir Widgets muy interesantes creados por la comunidad. Desde el mismo botón de Add Widget, pulsa en la pestaña de IoT Hub y busca “scatter plot”. Puedes instalarlo desde ahí como muestra la Figura 9. Aprovecha para configurarlo mostrando `touch_x` y `touch_y`, las variables sintéticas que se están enviando.

Tras haber añadido ambos widgets, deberías tener un panel similar a este, que se va actualizando automáticamente con los valores que llegan desde **32F723EDISCOVERY**.

¿Y... cómo se está haciendo toda esta magia detrás de las cámaras? Puedes analizar, viendo el código de `main.c` que cada cierto tiempo se llama a una función de envío de telemetría. Esta misma se reproduce a continuación:

```

1  /*
2  * Generic publish telemetry over MQTT
3  */
4  uint8_t ESP01S_SendTelemetry_MQTT(ESP01S_TypeDef * esp01s, float temp, uint32_t
    counter, uint16_t touch_x, uint16_t touch_y) {
5      uint8_t buf[256];
6      char payload[256];

```

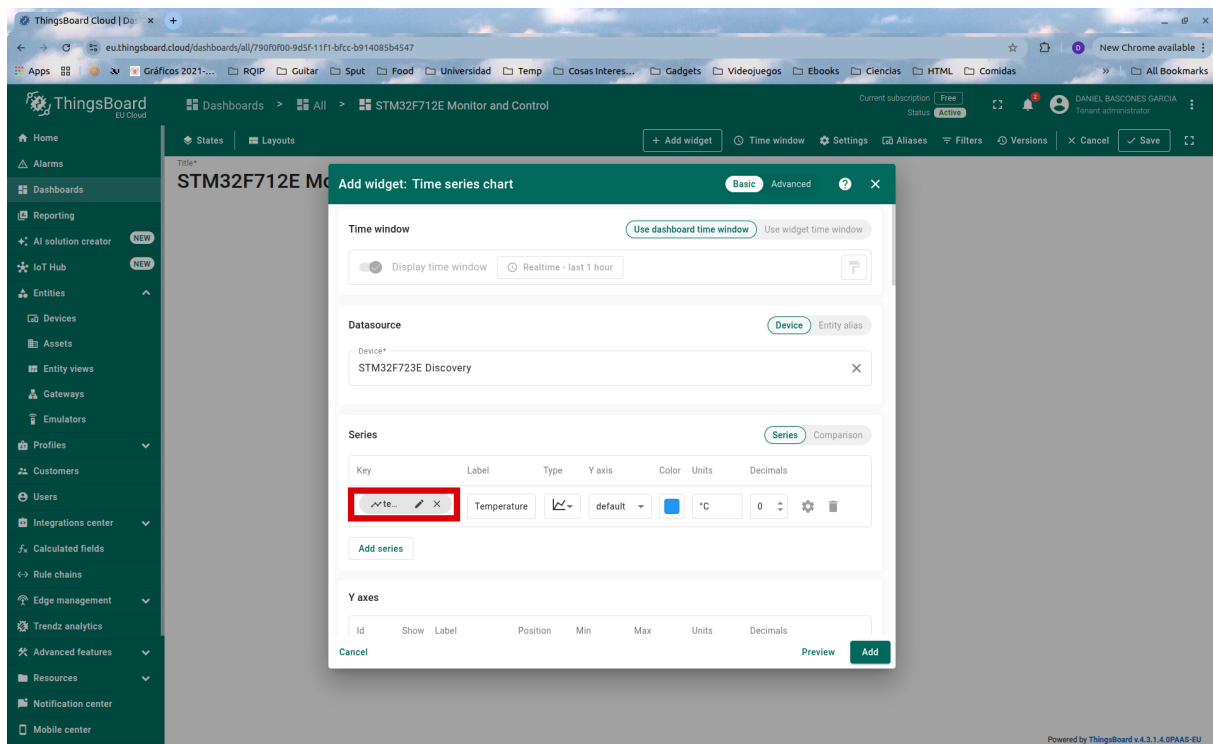


Figura 8: Añadiendo una gráfica que muestre la variable “temperature”

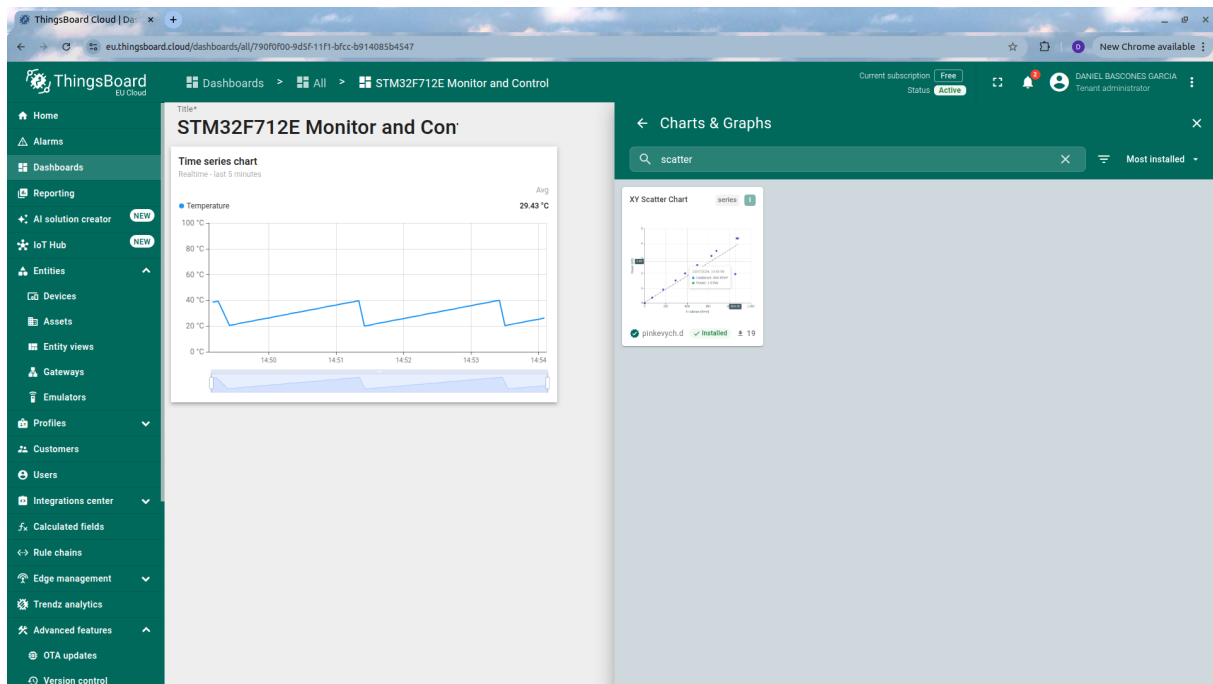


Figura 9: Instalando nuevos widgets

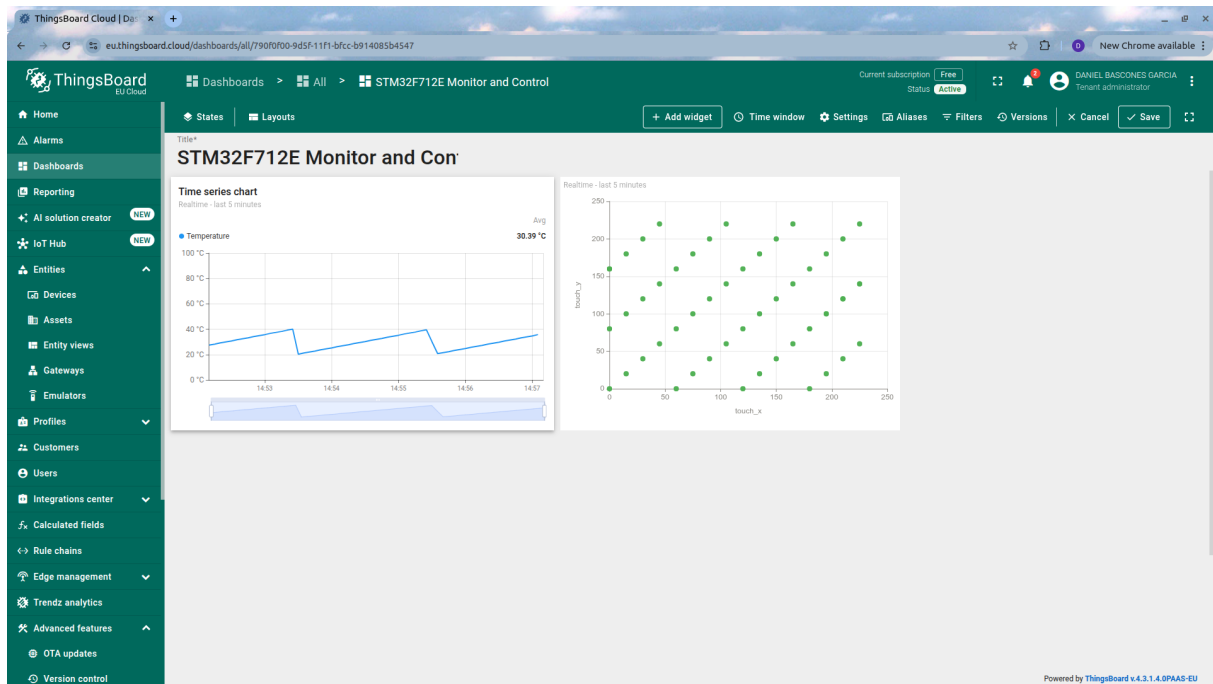


Figura 10: Panel finalizado

```

7
8     int temp_int = (int)temp;
9     int temp_frac = (int)(abs((int)(temp * 100.0f)) % 100);
10
11     snprintf(payload, sizeof(payload),
12              "{ \"temp\": %d.%02d, \"counter\": %lu, \"touch_x\": %u, \"touch_y\": %u }",
13              temp_int, temp_frac, (unsigned long)counter, touch_x, touch_y);
14
15     MQTTString topicString = MQTTString_initializer;
16     topicString.cstring = "v1/devices/me/telemetry";
17
18     int len = MQTTSerialize_publish(buf, sizeof(buf), 0, 0, 0, 0,
19                                   topicString, (unsigned char *)payload,
20                                   strlen(payload));
21
22     UART_WriteBytes(esp01s->UARTx, buf, len);
23     Debug_Printf("[MQTT TX] %s\r\n", payload);
24     return 1;
25 }

```

Puedes ver que primero se construye el *payload MQTT* o carga, con las variables a enviar. A continuación, se declara el *topic MQTT* o identificador donde enviar estas variables (por ejemplo, donde otros clientes podrían estar suscritos para escuchar cambios). El mensaje se forma con la librería incluida, mediante la función `MQTTSerialize_publish`, y se envía a través de la `UART` que comunica con el módulo ESP01S, que está configurada en modo *passthrough* para, básicamente, transformar nuestra `UART` en un canal TCP con el servidor.

Es en esta función en la que te tendrías que fijar si quisieras enviar cualquier otro tipo de dato, cambiando el *topic* y el *payload*.

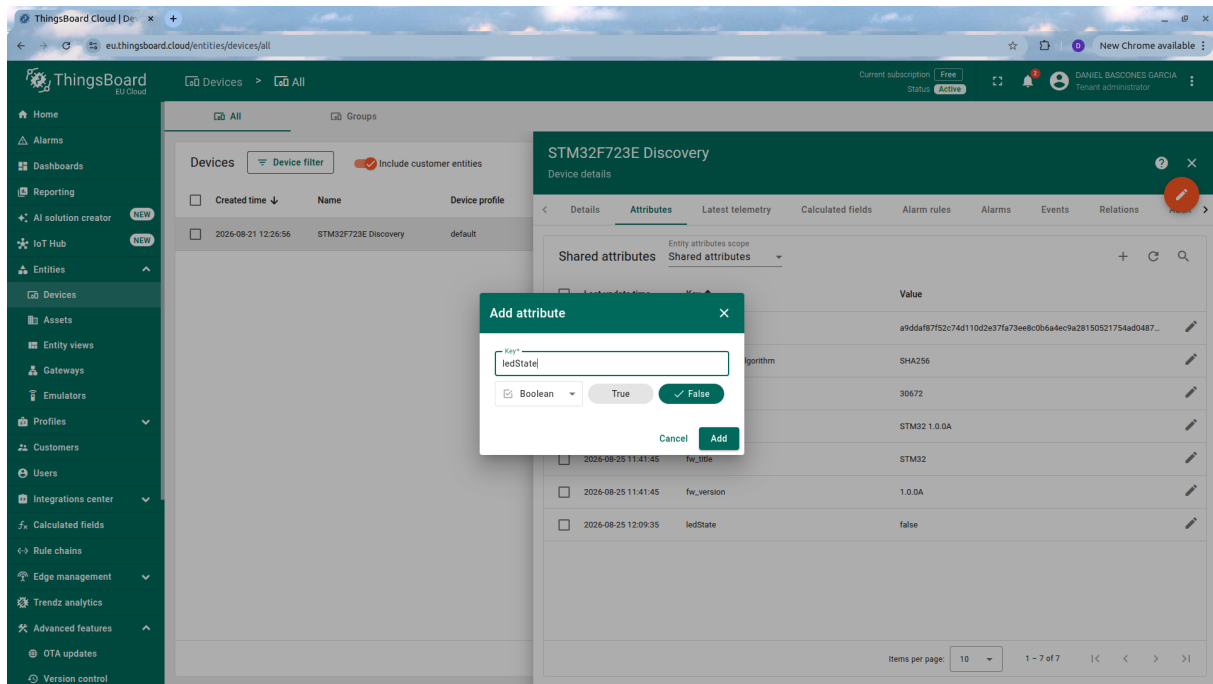


Figura 11: Atributo compartido para nuestro dispositivo

1.3.4. Recibiendo comandos desde *ThingsBoard*

Algo muy útil es la posibilidad no sólo de recibir información en *ThingsBoard*, sino también de enviar datos desde la misma plataforma, a fin de que los dispositivos se puedan configurar o accionar remotamente. En este caso, controlaremos el LED de la placa con un interruptor remoto. Para ello vamos a la configuración del dispositivo, y desde la pestaña “Attributes”, añadimos un nuevo atributo compartido (*shared*) llamado `ledState`, como en la Figura 11.

Ahora tenemos que añadir un botón que lo pueda gestionar. Añadimos a nuestro panel un widget “single switch”, y lo configuramos como muestra la Figura 12. Básicamente lo que estamos haciendo es añadir un botón que cambiará este valor asociado a nuestro dispositivo.

ThingsBoard, como *broker* que es, avisará al dispositivo cada vez que cambie este valor, ya que es un atributo compartido que debería actualizar. Si hacemos click, veremos que el LED ya cambia de color, pues la función de procesamiento de mensajes ya está hecha en la forma de `ESP01S.ProcessMQTT`, que se llama repetidamente desde el bucle de `main.c`:

Como es bastante larga, no la ponemos completa, pero consta de dos partes: en una primera, se extrae el paquete MQTT en crudo de la UART de entrada, y en una segunda, se parsea el paquete para extraer la información necesaria (básicamente, qué se ha actualizado):

```

1 // Paho Deserializer
2 unsigned char dup, retained;
3 int qos, payloadlen;
4 unsigned char* payload;
5 MQTTString receivedTopic;
6
7 if (MQTTDeserialize_publish(&dup, &qos, &retained, NULL, &receivedTopic,
8                             &payload, &payloadlen, packet, copy_len) == 1)
9 {

```

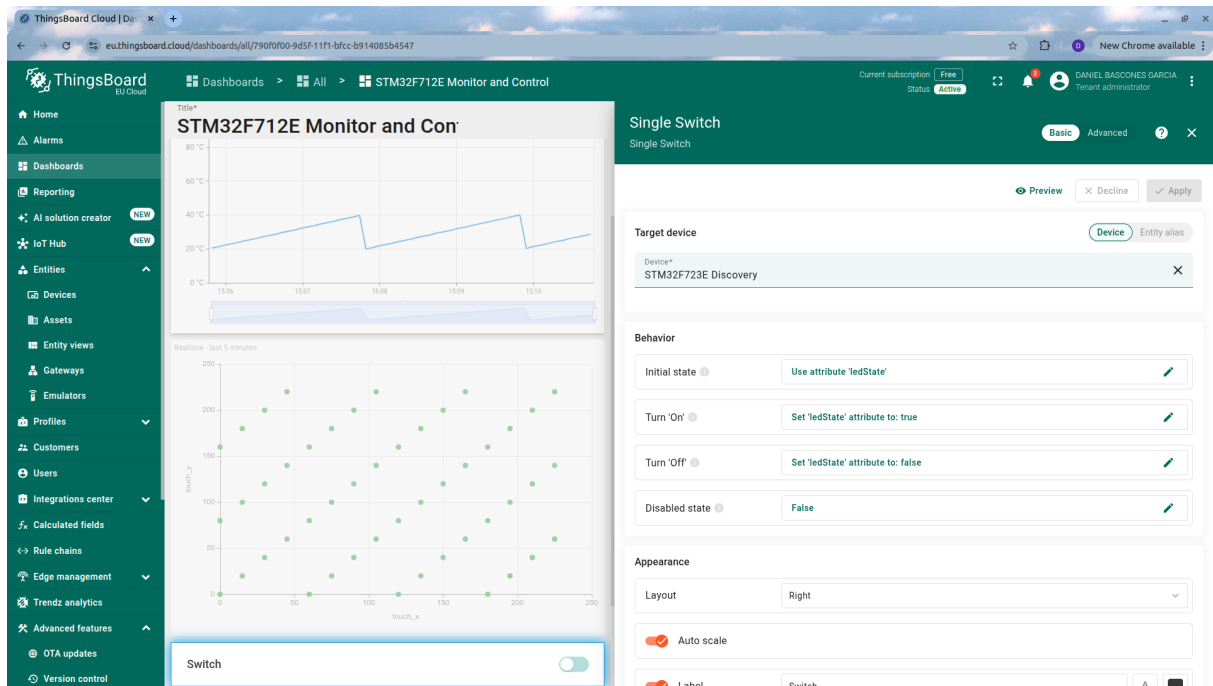


Figura 12: Atributo compartido para nuestro dispositivo

```

10  Debug_Printf("[MQTT RX] %s\r\n", receivedTopic.lenstring.data);
11
12  char json_str[128] = {0};
13  uint16_t str_len = (payloadlen < sizeof(json_str) - 1) ? payloadlen : sizeof
14    (json_str) - 1;
15  memcpy(json_str, payload, str_len);
16
17  // LED command processing
18  if (strstr(json_str, "\"ledState\":true"))      GPIO_PIN_WRITE(GPIOA, 7,
19    GPIO_STATE_ONE);
20  else if (strstr(json_str, "\"ledState\":false"))  GPIO_PIN_WRITE(GPIOA, 7,
    GPIO_STATE_ZERO);

```

Como se puede observar, si se recibe un mensaje conteniendo la cadena "ledState":true o "ledState":false, el LED cambiará su valor al recibido.

NOTA: En realidad, la forma canónica de hacer esto es utilizando la clase `MQTTClient` y registrando *callbacks* para atributos concretos recibidas, que llamarán a funciones que hagan la funcionalidad (valga la redundancia) asociada a dichos atributos. Sin embargo, la librería completa da algunos problemas de stack overflow. Si alguien se anima a optimizar esta práctica para que funcione (de manera consistente) con esa clase, es un buen proyecto final.

1.4. Para hacer más

Ahora que ya tienes la placa conectada con un servidor remoto, y puedes mandar datos y recibir comandos desde ella, puedes explorar las opciones de *ThingsBoard* en general o de los *dashboard* más concretamente, o hacer por ejemplo que los datos que se envían sean reales, en lugar de sintéticos.